

goVerifEye Swagger API Testing Guide

Last updated: July 22, 2026

Environments

Production/demo API:

- API base URL: <https://goverifeye-api.onrender.com/api/v1>
- Swagger UI: <https://goverifeye-api.onrender.com/docs>
- Liveness: <https://goverifeye-api.onrender.com/api/v1/health>
- Readiness: <https://goverifeye-api.onrender.com/api/v1/health/ready>

The Render Free service sleeps after inactivity. The first request can take approximately one minute. Open the readiness URL and wait for HTTP 200 before starting a test session.

Before testing

Confirm the deployed service has valid environment values for:

- Supabase database and Storage
- JWT access and refresh secrets
- Activation-code pepper
- Gmail OAuth credentials
- Allowed frontend origins
- Private Storage bucket names

Create the private Supabase buckets configured by:

- `SUPABASE_STORAGE_BUCKET` (default: `compliance-documents`)
- `SUPABASE_ARTIFACT_BUCKET` (default: `generated-artifacts`)

Do not use real customer information. Use dedicated test organizations, products, email addresses, and documents.

How to execute a Swagger request

1. Open the Swagger UI.
2. Expand an endpoint.
3. Select **Try it out**.
4. Enter path/query values and the JSON request body.
5. Add required headers such as `Idempotency-Key` where Swagger displays them.
6. Select **Execute**.
7. Record the response status, body, and `x-correlation-id` response header.

Protected endpoints return HTTP 401 until a JWT is supplied.

Authentication and authorization

1. Request a registration OTP

Execute:

```
POST /api/v1/auth/otp/request
```

Body:

```
{
  "email": "vendor-test@example.com"
}
```

Expected response: HTTP 201 with **challengeId** and **expiresInSeconds**. In production, the OTP is delivered through Gmail and is not returned by the API.

2. Verify the OTP

Execute:

```
POST /api/v1/auth/otp/verify
```

```
{
  "email": "vendor-test@example.com",
  "code": "123456"
}
```

Expected response: HTTP 200 with a short-lived **registrationToken**.

3. Register

Execute:

```
POST /api/v1/auth/register
```

```
{
  "registrationToken": "paste-registration-token",
  "password": "StrongTest1Password"
}
```

Expected response: access and refresh tokens. Copy both values immediately.

4. Authorize Swagger

1. Select **Authorize** near the top of Swagger.
2. Enter only the access token unless the dialog explicitly requests the full scheme.
3. If required by the displayed input, enter **Bearer ACCESS_TOKEN**.
4. Select **Authorize**, then close the dialog.
5. Execute **GET /api/v1/auth/me**.

Expected response: HTTP 200 containing the authenticated database user. The server derives **userId**, **organizationId**, session, and role from the JWT; clients must never submit them as actor fields.

5. Login and refresh

Login:

```
POST /api/v1/auth/login
```

```
{
  "email": "vendor-test@example.com",
  "password": "StrongTest1Password"
}
```

Refresh:

```
POST /api/v1/auth/refresh
```

```
{
  "refreshToken": "paste-refresh-token"
}
```

Refresh tokens rotate. Replace the old refresh token after every successful refresh. Reusing an old token should return HTTP 401 and revoke the affected session.

Recommended end-to-end test order

A. Authentication/session tests

1. Register and authorize Swagger.
2. Execute **GET /auth/me**.
3. Execute the paginated **GET /auth/sessions**.
4. Refresh the token and re-authorize Swagger.
5. Confirm old refresh-token reuse is rejected.
6. Revoke a secondary session with **DELETE /auth/sessions/{id}**.

7. Test `POST /auth/logout` last because it invalidates the current session.

B. Onboarding

Save each draft section:

```
PATCH /onboarding/company
PATCH /onboarding/administrator
PATCH /onboarding/address
```

Example company body:

```
{
  "companyName": "Swagger Test Industries Ltd",
  "registrationNumber": "TEST-RC-10001",
  "industry": "Consumer goods",
  "country": "Nigeria"
}
```

Example administrator body:

```
{
  "firstName": "Ada",
  "lastName": "Tester",
  "email": "vendor-test@example.com",
  "phone": "+2348012345678"
}
```

Example address body:

```
{
  "line1": "1 Test Avenue",
  "city": "Lagos",
  "state": "Lagos",
  "country": "Nigeria",
  "postalCode": "100001"
}
```

C. Compliance-document upload

1. Prepare a small valid PDF, PNG, or JPEG.
2. Determine its exact byte size.
3. Execute `POST /onboarding/documents`:

```
{
  "type": "company_registration",
  "fileName": "registration.pdf",
  "mimeType": "application/pdf",
  "size": 12345
}
```

4. Copy the signed upload URL/token from the response.
5. Upload the raw file using a REST client or the Supabase signed-upload method. Swagger creates the upload authorization but does not send the binary file itself.
6. Execute `POST /onboarding/documents/{id}/complete`.
7. The server downloads the object, validates size and file signature, calculates SHA-256, and invokes the configured malware scanner.
8. Confirm the document status becomes `verified`.
9. Test `GET /onboarding/documents/{id}/download` and confirm the signed download expires.
10. Execute `POST /onboarding/submit` only after at least one document is verified.

If `MALWARE_SCAN_REQUIRED=true` and no scanner is configured, completion should fail closed with HTTP 503.

D. Platform review setup

Vendor registration creates an organization administrator, not a platform administrator. Full approval testing requires a separate controlled test account with database role:

```
platform_admin
```

Create two test accounts. In the Supabase SQL Editor, promote only the dedicated reviewer account:

```
update users
set role = 'platform_admin'
where email = 'reviewer-test@example.com';
```

Log in again after changing the role so the session uses the current database authorization. Never promote a vendor account in production without an approved operational process.

Using the reviewer JWT:

1. Execute `GET /platform/approvals/queue?resourceType=onboarding`.
2. Execute `POST /platform/approvals/onboarding/{organizationId}/decision`.

```
{
  "decision": "approved",
  "notes": "Test onboarding evidence accepted"
}
```

3. Confirm the review in `GET /platform/approvals/onboarding/{id}/history`.

E. Products and product approval

Using the vendor JWT, create a product:

`POST /products`

```
{
  "name": "Swagger Test Product",
  "description": "A dedicated non-production product for API validation.",
  "form": "Packaged item",
  "manufacturer": "Swagger Test Industries Ltd"
}
```

The product should start as `pending`.

Using the platform-reviewer JWT:

1. Execute `GET /platform/approvals/queue?resourceType=product`.
2. Execute `POST /platform/approvals/products/{productId}/decision`:

```
{
  "decision": "approved",
  "notes": "Test product approved"
}
```

Using the vendor JWT again, verify:

- `GET /products`
- `GET /products/{id}`
- `PATCH /products/{id}`
- archive, restore, reject/resubmit lifecycle where applicable
- filtering, sorting, pagination, and cross-tenant isolation

F. Cryptographic code generation

Code generation requires an active product and an `Idempotency-Key` header.

Execute:

```
POST /code-batches
Idempotency-Key: swagger-batch-0001
```

```
{
  "productId": "approved-product-uuid",
  "labelType": "pair",
  "fulfillment": "selfprint",
  "paperSize": "A4",
  "quantity": 100
}
```

Expected response: a generated batch and 100 one-time credential pairs containing:

- A 16-digit numeric verification code
- A separate activation code

Save the response securely. Raw activation codes are not stored and cannot be recovered.

Repeat the identical request with the same **Idempotency-Key**. Expected behavior:

- No second batch is created.
- The existing batch is returned.
- Activation credentials are not returned again.

Test:

- **GET /code-batches** with page, pageSize, sort, status, product, and search filters
- **GET /code-batches/{id}**
- **GET /code-batches/{id}/codes**
- code suspend/reactivate lifecycle

G. Activate and publicly verify a code

Activate using one saved pair:

```
POST /code-batches/codes/activate
```

```
{
  "verificationCode": "1234567890123456",
  "activationCode": "12345678"
}
```

Public verification does not require Swagger authorization:

```
POST /code-batches/codes/verify
```

```
{
  "verificationCode": "1234567890123456",
  "location": "Lagos, Nigeria"
}
```

Test first verification, repeat verification, rapid repeated verification, inactive, suspended, unknown, and malformed codes. Rapid/repeated verification should contribute to risk scoring and suspicious-event reporting without storing raw client IP addresses.

H. Reporting

Test these with the vendor JWT:

- GET /dashboard
- GET /reports
- GET /reports/top-products
- GET /reports/suspicious-scans
- GET /reports/verification-events
- GET /reports/locations
- GET /reports/trends
- GET /reports/funnel
- product-specific activity and verification endpoints

Example query:

```
?page=1&pageSize=20&sortBy=createdAt&sortDirection=desc&from=2026-07-01T00:00:00.000Z&to=2026-07-31T23:59:59.999Z&productId=PRODUCT_UUID
```

Verify invalid dates, invalid sort fields, oversized page sizes, and another tenant's product ID are rejected or return no tenant data.

I. Team invitations

Using an organization-admin JWT:

1. Execute POST /team/members/invite.
2. Confirm the production response does not expose the invitation token.
3. Confirm Gmail delivers the invitation.
4. Test invitation summary and acceptance without vendor authorization.
5. Test paginated invitation listing, resend, revoke, member update, deactivate, and reactivate.
6. Confirm an administrator cannot deactivate themselves or the final active administrator.

7. Confirm staff receives HTTP 403 for administrator-only operations.

J. Notifications, audit, settings, jobs, and support

Test:

- Notification list, unread count, mark-one/read-all, and delete
- Audit filters, sorting, pagination, and admin-only access
- Profile/company settings
- Password change and old-password rejection
- Session revocation
- Account deactivation final-admin protection
- Export/print job creation using a unique **Idempotency-Key**
- Job status polling and signed artifact result
- Vendor support-ticket creation/listing
- Platform-admin support queue/status updates

Payment behavior

Payment is intentionally deferred. Execute:

```
POST /payments
```

Expected response: HTTP 501 with an **application/problem+json** response. The API must not simulate successful payment or trigger paid fulfilment.

Negative and security tests

For each protected resource, test:

- No Authorization header → HTTP 401
- Expired or malformed JWT → HTTP 401
- Revoked session → HTTP 401
- Staff calling admin route → HTTP 403
- Vendor calling platform route → HTTP 403
- Resource ID owned by another organization → HTTP 404 or an empty tenant-scoped result
- Unknown JSON property → HTTP 400
- Invalid UUID, enum, date, numeric range, or sort field → HTTP 400
- Missing/short **Idempotency-Key** → HTTP 400 where required
- Reused idempotency key for another operation → HTTP 409
- Excessive auth/public-verification requests → HTTP 429
- Invalid resource state transition → HTTP 409

Error responses should use:

```
Content-Type: application/problem+json
```

Example:

```
{
  "type": "https://api.goverifeye.com/problems/product_not_found",
  "title": "Product was not found",
  "status": 404,
  "code": "PRODUCT_NOT_FOUND",
  "instance": "/api/v1/products/...",
  "correlationId": "..."
}
```

Collection contract checklist

For every list endpoint, verify:

- `page` starts at 1
- `pageSize` is bounded
- only allowlisted `sortBy` fields work
- `sortDirection` accepts only `asc` or `desc`
- supported search/status/date/product filters work
- response `meta` contains page, pageSize, total, totalPages, hasNextPage, and hasPreviousPage
- no record from another organization is returned

Test evidence template

Record this for every release:

Field	Value
Environment	
API commit	
Test date/time	
Tester	
Endpoint	
Role	
Request summary	
Expected status	
Actual status	
Correlation ID	
Result	Pass / Fail
Defect reference	

Never store passwords, JWTs, refresh tokens, OTPs, invitation tokens, activation codes, Supabase keys, Gmail credentials, or database connection strings in test evidence.